Image Classification on CIFAR-10: Effect of ConvNet Architecture

Anugrah Putra Al Fatih (26/590120/PPA/07385)

GitHub: https://github.com/anugrahhhhh/CNN-Image-Classification-2

1. Objective and Setup

The provided notebook (*colab img_clf.ipynb*) was executed and its **ConvNet** (3 conv blocks + 3 fully connected blocks) was used as the starting point. Several architectural variants were then trained under **identical** conditions so that only the architecture differs between runs. Settings kept from the notebook: CIFAR-10, training set = *data_batch_1* (10,000 images), validation = *data_batch_2* (10,000), test = *test_batch* (10,000); input scaled by /255; sigmoid output with binary cross-entropy on one-hot labels; SGD (lr = 0.0025, momentum = 0.9); batch size 10. Changes for a fair comparison: (i) **40 epochs** instead of 1, because the from-scratch ConvNet barely moves away from chance in the first epochs; (ii) the checkpoint with the best validation accuracy is evaluated on the test set; (iii) fixed seed (0); (iv) the code runs locally on CPU (PyTorch CPU build, 8 threads) instead of the Colab GPU.

Bug found in the notebook data loader. In *myData.__getitem__* all three channels are sliced with *[0:1024]*, so the network receives the red channel copied three times instead of RGB. Run E0 reproduces this bug; all other runs use the corrected slices (R = 0:1024, G = 1024:2048, B = 2048:3072).

2. Architectures Compared

The notebook ConvNet follows the pattern *3 × conv_block → 3 × fc_block*, where conv_block = Conv + ReLU + MaxPool(2) and fc_block = Linear + ReLU (sigmoid at the output). Table 1 lists the layers of the baseline (E1) next to the best variant (E7). Each experiment changes **one** design factor so its effect can be isolated: **width** (E2, more filters per layer), **depth** (E3, a 4th conv block; padding = 1 so the map does not shrink to zero), **receptive field** (E4, 5×5 kernels), the **feature aggregation head** (E5, flatten the spatial map instead of global average pooling) and **normalisation** (E6, BatchNorm after every conv). E7 then combines the factors that helped, plus Dropout 0.3 in the FC layers to fight overfitting.

Block	E1 baseline layer	Output	Params	E7 best layer	Output	Params
Input	-	3×32×32	-	-	3×32×32	-
Conv 1	Conv3×3 3→16, ReLU, MaxPool	16×15×15	448	Conv3×3 p1 3→32, BN, ReLU, Pool	32×16×16	960
Conv 2	Conv3×3 16→32, ReLU, MaxPool	32×6×6	4,640	Conv3×3 p1 32→64, BN, ReLU, Pool	64×8×8	18,624
Conv 3	Conv3×3 32→64, ReLU, MaxPool	64×2×2	18,496	Conv3×3 p1 64→128, BN, ReLU, Pool	128×4×4	74,112
Head	Global average pool	64	0	Flatten	2048	0
FC 1	Linear 64→128, ReLU	128	8,320	Linear 2048→128, ReLU, Dropout	128	262,272
FC 2	Linear 128→64, ReLU	64	8,256	Linear 128→64, ReLU, Dropout	64	8,256
FC 3	Linear 64→10, Sigmoid	10	650	Linear 64→10, Sigmoid	10	650
Total			40,810			364,874

Table 1. Layer-by-layer comparison of the baseline ConvNet (E1) and the best variant (E7). Most E7 parameters sit in FC1 because flattening keeps the 128×4×4 spatial map.

3. Results

ID	Architecture change (vs. E1)	Params	Best epoch	Val acc (%)	Test acc (%)	Macro F1	Train time (s)	Latency (ms/img)
E0	Notebook as-is (loader copies R channel to G,B)	40,810	40	16.8	16.9	0.115	119	0.28
E1	Baseline 3 conv (16-32-64) k3, GAP, FC 128-64-10	40,810	40	25.9	26.2	0.201	131	0.27
E2	Wider: 32-64-128 filters	118,666	40	28.8	29.4	0.247	178	0.37
E3	Deeper: 4 conv blocks (16-32-64-128), padding=1	122,858	39	34.7	36.2	0.346	176	0.34
E4	Kernel 5x5 (padding=2)	82,538	39	32.2	33.4	0.314	169	0.57
E5	Flatten 64x4x4 instead of global avg pool	163,690	40	51.6	51.9	0.509	142	0.29
E6	Baseline + BatchNorm	41,034	31	59.0	59.6	0.595	173	0.40
E7	Wide + pad + BN + Flatten + Dropout 0.3	364,874	22	66.7	**67.3**	**0.668**	290	0.57

Table 2. Results of all runs (40 epochs, same optimizer, loss, data and seed). Val/test accuracy use the best-validation checkpoint. Macro F1 = unweighted mean of per-class F1 on the test set. 'Flatten' (E5) flattens the 64×4×4 map (padding = 1). Latency = single-image forward pass on CPU (batch 1, as in the notebook test loop).

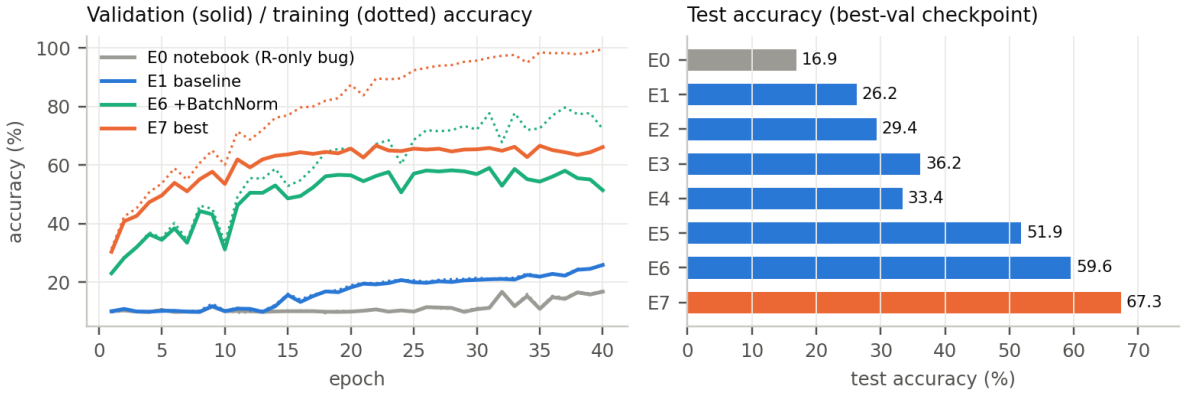


Figure 1. Left: learning curves for the notebook version (E0), the corrected baseline (E1), baseline + BatchNorm (E6) and the best model (E7). Right: test accuracy of all variants.

4. Analysis

4.1 Effect of each architectural change

- **Correct input data matters before architecture.** Fixing the RGB loader alone raised test accuracy from 16.9% (E0) to 26.2% (E1) with the same network. With the bug, colour cues (blue sky/sea for airplane/ship, green background for frog/deer) are lost.
- **The baseline underfits.** E1 stays at ~10% (chance) for about 12 epochs and ends at only 26.0% *training* accuracy, the same as its validation accuracy. So the problem is optimisation and capacity, not overfitting. A sigmoid + BCE output with a small learning rate gives weak gradients at the start (all outputs near 0.1). The tiny 2×2 final map is also averaged into only 64 features.
- **Width vs. depth vs. kernel size.** Doubling the filters (E2, 2.9× params) gave only +3.1 pp. Adding a 4th conv block with padding (E3) gave +9.9 pp, and 5×5 kernels (E4) +7.1 pp. Larger receptive fields and keeping spatial resolution (padding) help more than simply adding filters.
- **GAP vs. Flatten.** Replacing global average pooling with flattening (E5) gave the largest single gain among the 'plain' changes: 51.9% (+25.6 pp). GAP throws away *where* a feature appears. That is costly for a shallow 3-layer network whose features are not yet very abstract. The price is 4× more parameters (mostly in FC1) and the first sign of overfitting (train-val gap 5 pp).
- **BatchNorm is the most effective change.** Adding BN (E6) costs only 224 extra parameters but raised test accuracy to 59.6% (+33.3 pp). It passed 25% validation accuracy at epoch 2 instead of epoch 40. BN normalises each layer's activations, which removes the poor-scaling problem of the un-centred /255 input and lets SGD make real progress. However, E6 overfits (train 72.3% vs. val 51.4% at epoch 40), and its validation curve is noisy.
- **Best model (E7)** combines the helpful changes (wider 32-64-128 + padding + BN + Flatten + Dropout 0.3). It reaches **67.3%** test accuracy, +41.1 pp over the baseline and +50.5 pp over the original notebook. Its training accuracy reaches 99.6% while validation plateaus at ~65% from epoch ~20 (best epoch 22). With only 10,000 training images and 365k parameters, dropout alone is not enough regularisation.

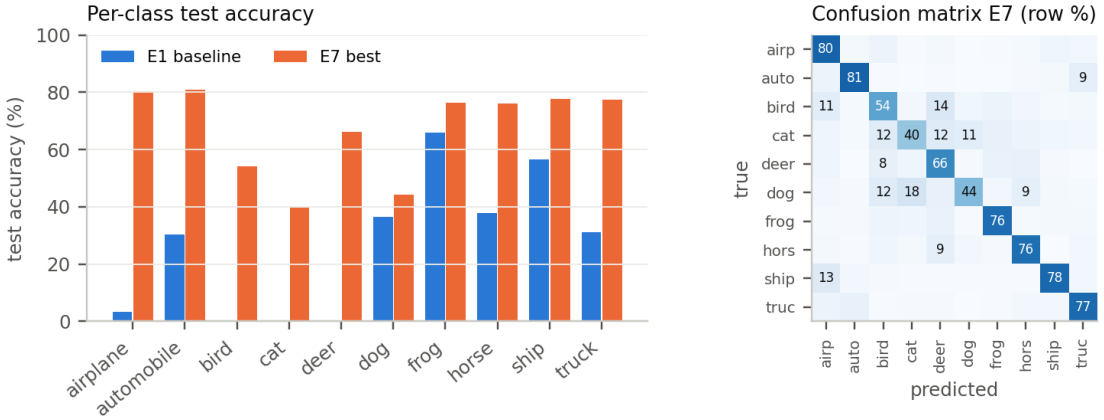


Figure 2. Left: per-class test accuracy of the baseline (E1) and best model (E7). Right: row-normalised confusion matrix of E7 (only cells ≥ 8% are labelled).

4.2 Per-class behaviour

- **Vehicles are easiest** (automobile 81%, airplane 80%, ship 78%, truck 77%). **Animals are hardest** (cat 40%, dog 44%, bird 54%). The main confusions are between visually similar classes: dog→cat (18% of dogs),

cat→bird/deer/dog (11-12%), bird→deer (14%), ship→airplane (13%) and automobile→truck (9%). These are classes with similar shape and background at 32×32 resolution.

ID	plane	auto	bird	cat	deer	dog	frog	horse	ship	truck	Mean
E0	0	17	5	0	0	**54**	50	1	10	32	16.9
E1	3	30	0	0	0	37	66	38	56	31	26.2
E2	16	52	2	1	5	38	62	38	50	29	29.4
E3	40	56	11	10	30	37	41	50	51	34	36.2
E4	32	60	14	16	22	21	36	48	65	21	33.4
E5	52	75	29	28	45	38	68	63	72	48	51.9
E6	65	73	51	**51**	46	41	73	51	**82**	62	59.6
E7	**80**	**81**	**54**	40	**66**	44	**76**	**76**	78	**77**	67.3

Table 3. Per-class test recall (%) of every variant. Bold = best per class; red = class almost never predicted correctly (< 5%).

- Table 3 shows that the improvements are **broad, not class-specific**. The baseline E1 has collapsed onto a few classes (bird, cat, deer < 1% recall); from E3/E4 onward these classes start to be recognised. BatchNorm (E6) improves *every* class over E1, and E7 is the best model on 7 of 10 classes (E6 wins cat and ship). *Cat* and *dog* are the two weakest classes of both strong models (E6, E7) and are the main source of the remaining error. E0's high dog recall (54%) is an artefact: without colour the model predicts 'dog' for a large share of all images.

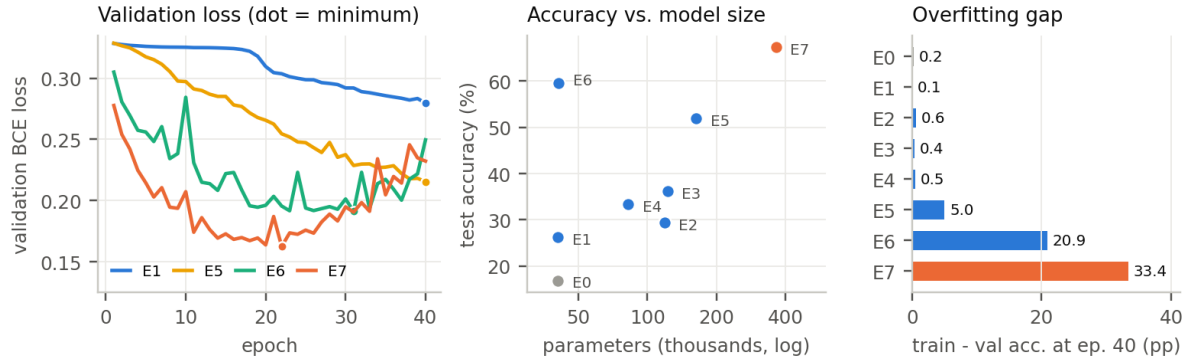


Figure 3. Left: validation BCE loss (dot = epoch of minimum loss). Middle: test accuracy vs. number of parameters (log scale). Right: overfitting gap = training minus validation accuracy at the last epoch.

4.3 Training dynamics, overfitting and calibration

- The **validation loss** separates the models better than accuracy. E1 and E5 are still decreasing at epoch 40 (more epochs would help). E6 and E7 reach their minimum early (epoch 31 and 22) and then *rise* again (E7: 0.162 → 0.232). At the same time their validation accuracy stays flat. The networks keep becoming **more confident on the wrong predictions** (worse calibration) even though the ranking of classes barely changes. Early stopping on validation loss would pick a better-calibrated model.
- The **overfitting gap** grows with capacity and ease of optimisation: ≈0 pp for E0-E4 (underfitting), 5 pp for E5, 21 pp for E6 and 33 pp for E7. Accuracy is **not** a simple function of parameter count (Fig. 3, middle). E6 (41k) beats E2, E3 and E5, which have 3-4× more parameters. *How* the parameters are used (normalisation, spatial head) matters more than how many there are.

4.4 Computational cost and efficiency

- Training time on CPU grew from 131 s (E1) to 290 s (E7), and single-image latency from 0.27 to 0.57 ms (all far below 1 ms per image). E6 gives the best accuracy-per-parameter trade-off: it matches the baseline size (41k params) at more than double the accuracy. For reference, the notebook's EfficientNet-B0 transfer-learning run (Colab GPU, 1 epoch, same RGB bug) reported 27.9% test accuracy with ~4M parameters.

5. Limitations

Each configuration was trained once (seed 0), so differences of 1-3 pp (e.g. E2 vs. E4) are within the range that run-to-run noise could produce. Only one training batch (10k images) is used, as in the notebook, which exaggerates overfitting for the larger models. Hyper-parameters were not re-tuned per architecture, which disadvantages slow learners such as E1 (still improving at epoch 40).

6. Conclusion

Changing the ConvNet architecture changed test accuracy from 26.2% to 67.3% under an identical training budget. Ranked by impact: (1) **Batch Normalisation** (fixes the slow, unstable optimisation); (2) **keeping spatial information** (Flatten instead of GAP, padding); (3) **depth / larger receptive field**; (4) **width** (least gain per parameter). Because the larger models overfit the 10k-image training set, next steps would be data augmentation (random crop/flip), using all 5 training batches, weight decay, a learning-rate schedule, and softmax + cross-entropy instead of sigmoid + BCE.